CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

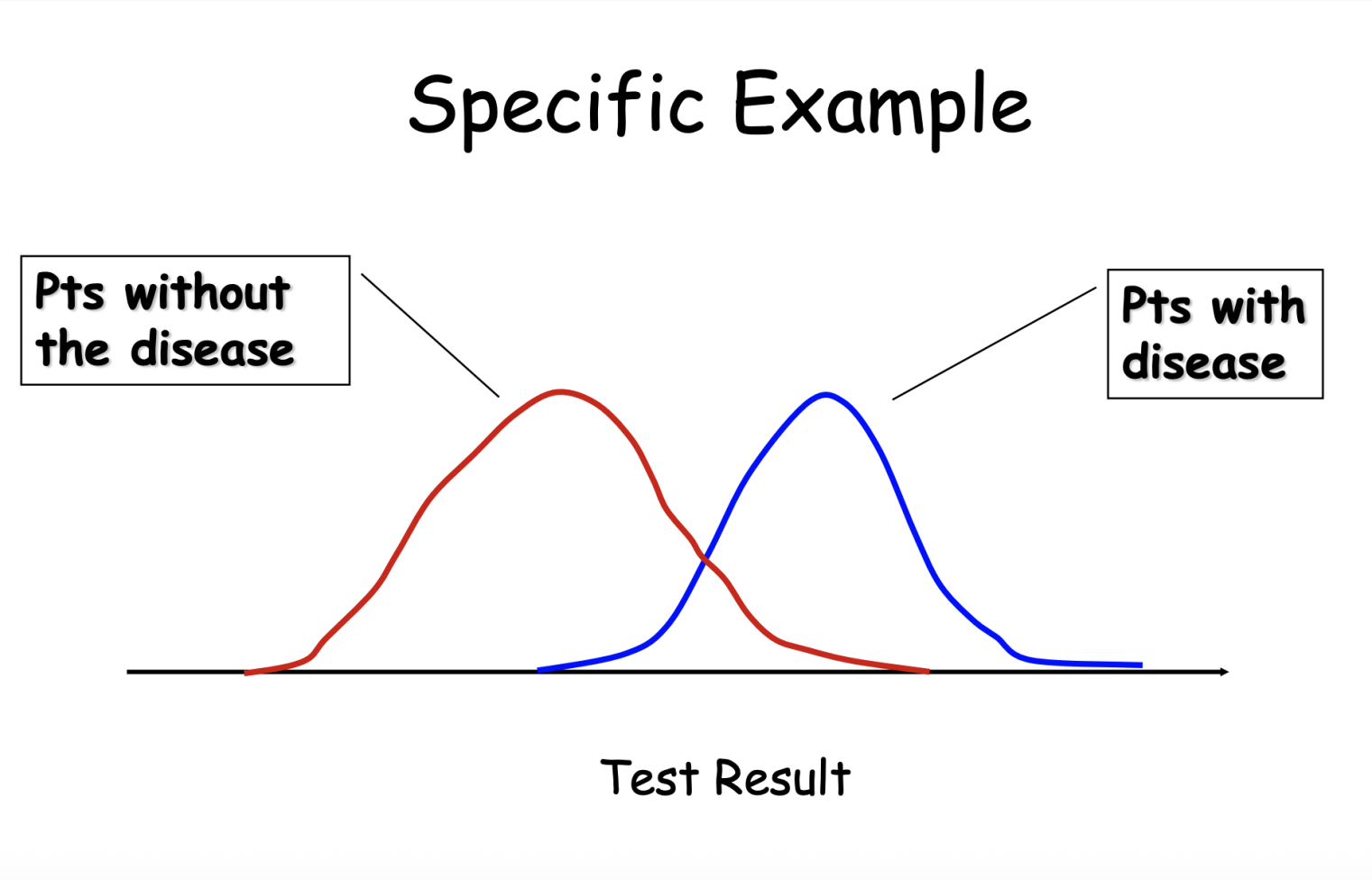
September 11, 2025

Lecture 7

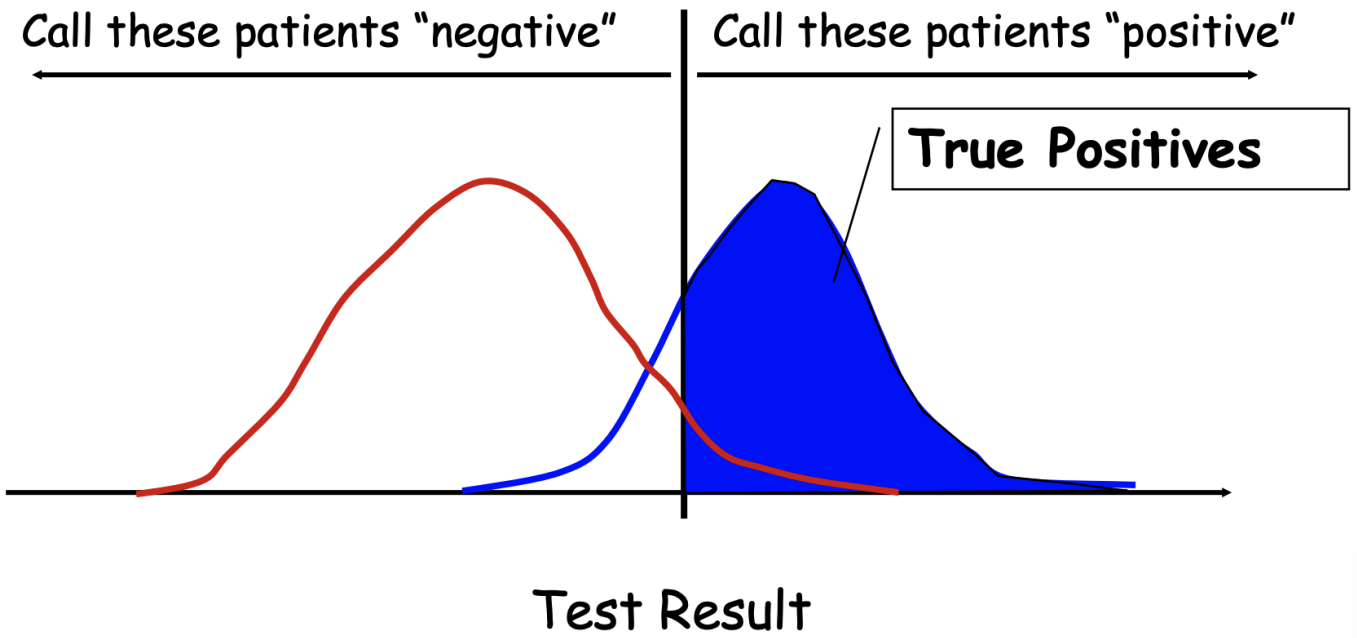
ROC Analysis: Medical Test Evaluation

- **True Positives = Test states you have the disease when you do have the disease (Good Result: No wasted Resources)**
- **True Negatives = Test states you do not have the disease when you do not have the disease (Good Result: No wasted resources)**
- **False Positives = Test states you have the disease when you do not have the disease (No Disease: wasted resources on future tests and potential harm from unnecessary treatment and increased anxieties)**
- **False Negatives = Test states indicated no disease when disease is actually present (Disease present but undiagnosed and untreated: very bad outcome)**

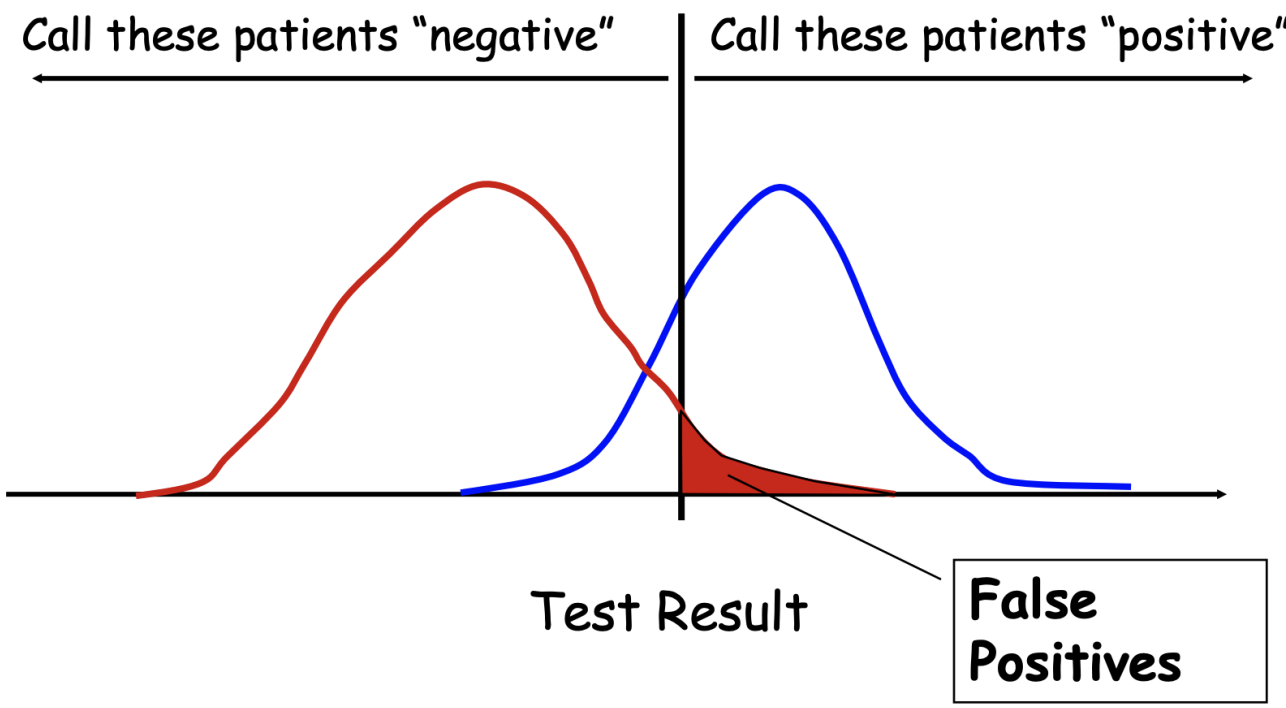
ROC Curve


Specific Example
Pts without the disease
Pts with disease
Test Result

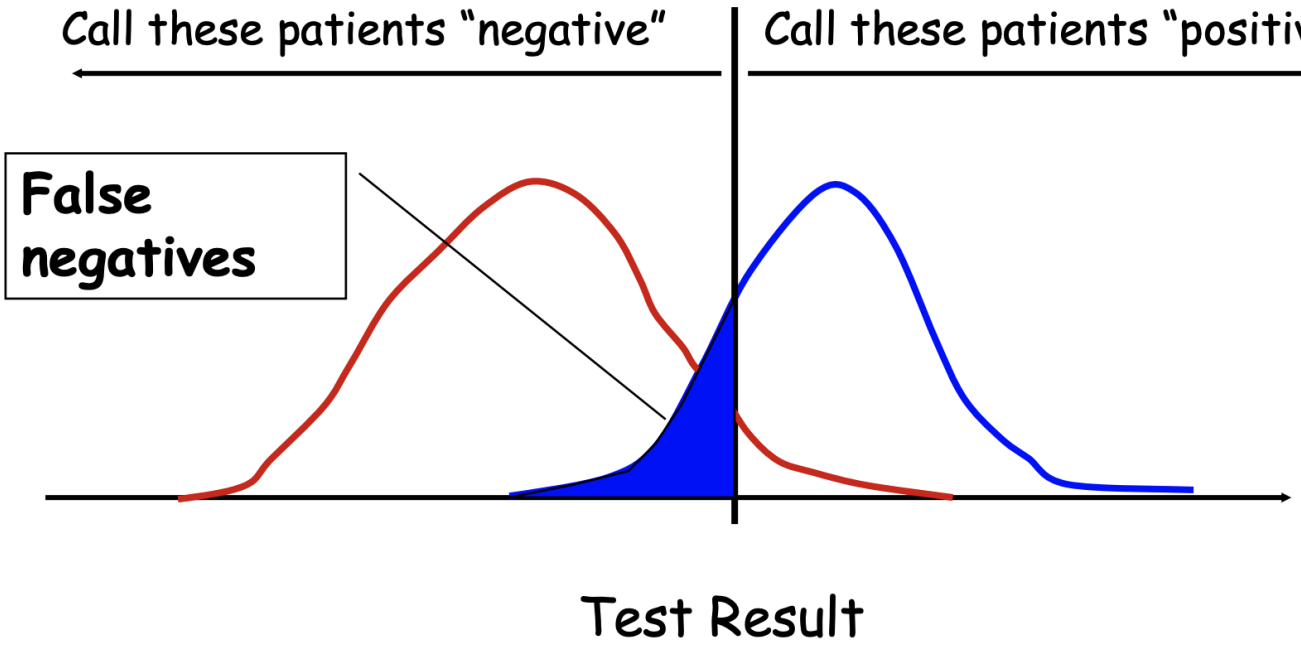
Some definitions ...



ROC Curve

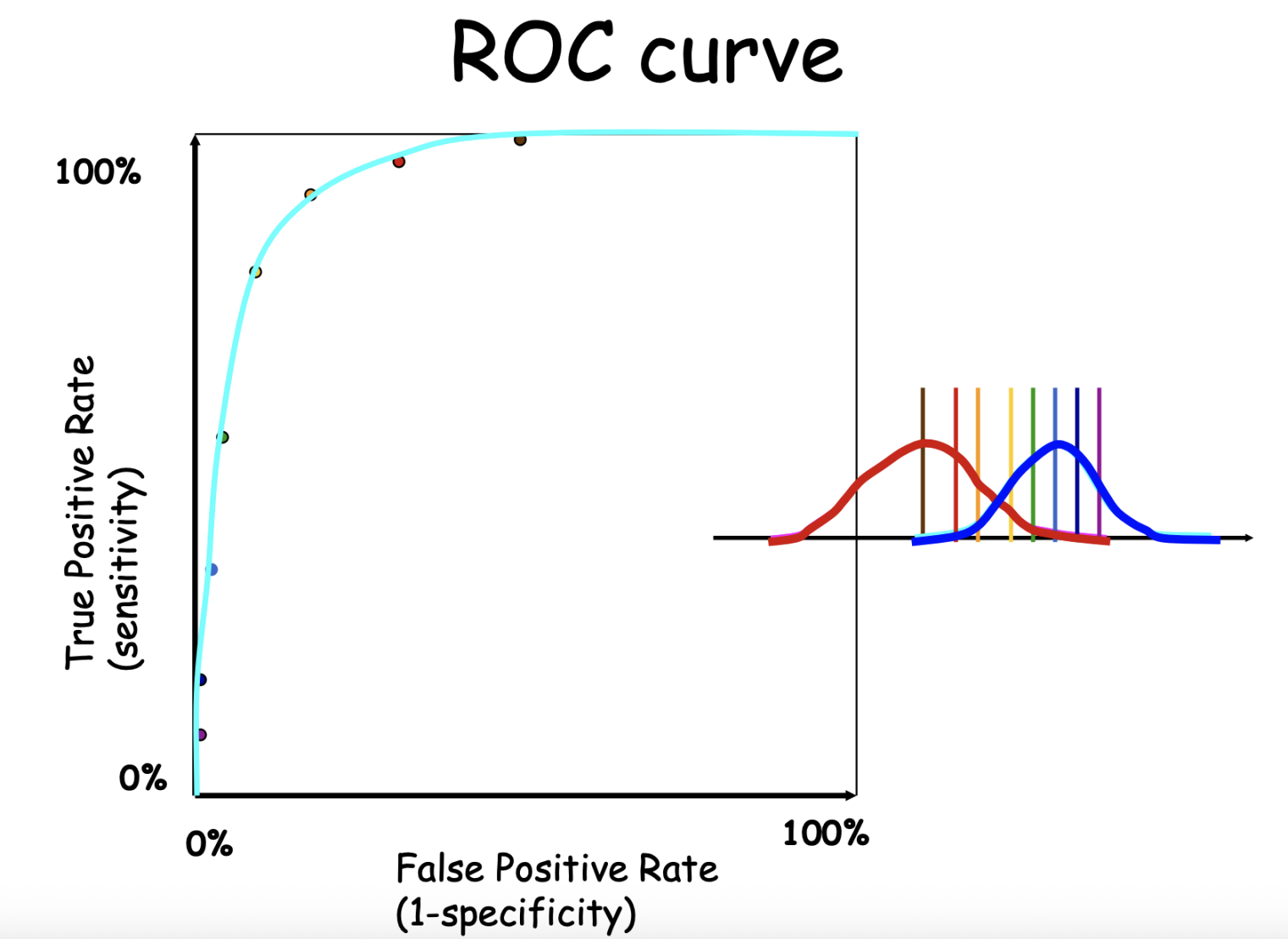

Call these patients "negative"
Call these patients "positive"
Test Result
False Positives
without the disease
with the disease

ROC Curve



without the disease
with the disease

ROC Curve

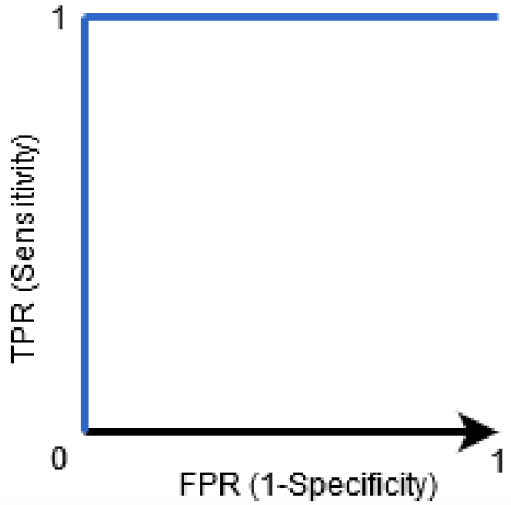

ROC curve
100%
True Positive Rate
(sensitivity)
0%
0%
100%
False Positive Rate
(1-specificity)

ROC Curve

- The Receiver Operator Characteristic (ROC) curve is an evaluation metric for binary classification problems.
- It is a probability curve that plots the TPR against FPR at various threshold values and essentially separates the ‘signal’ from the ‘noise’.
- The Area Under the Curve (AUC) is the measure of the ability of a classifier to distinguish between classes and is used as a summary of the ROC curve.

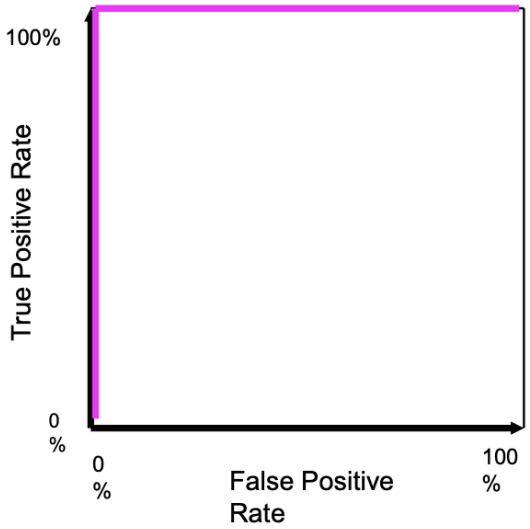
ROC Curve

- The higher the AUC, the better the performance of the model at distinguishing between the positive and negative classes.



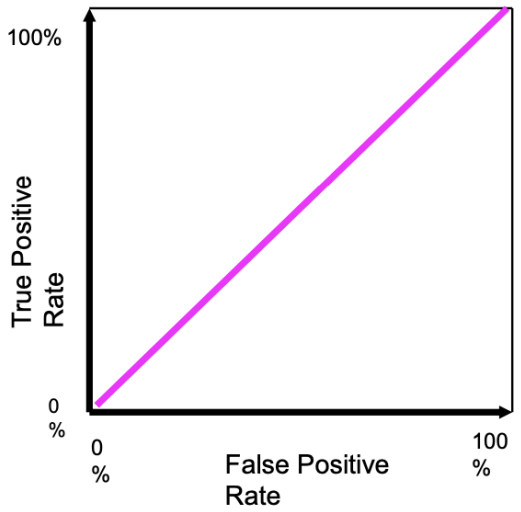
ROC curve extremes

Best Test:



The distributions don't overlap at all

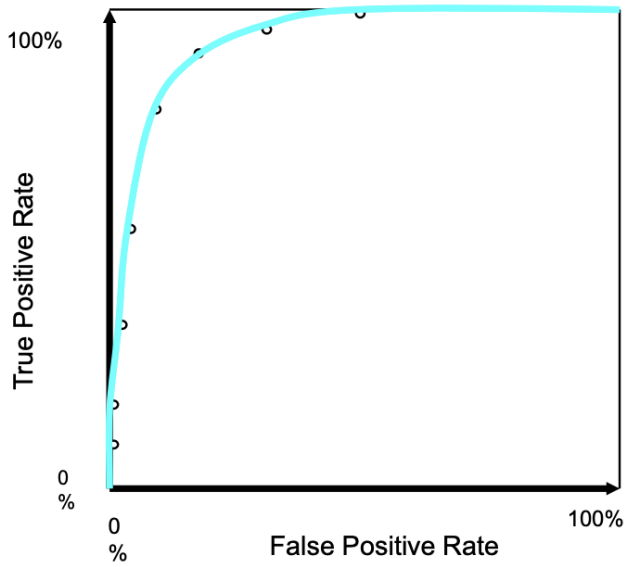
Worst test:



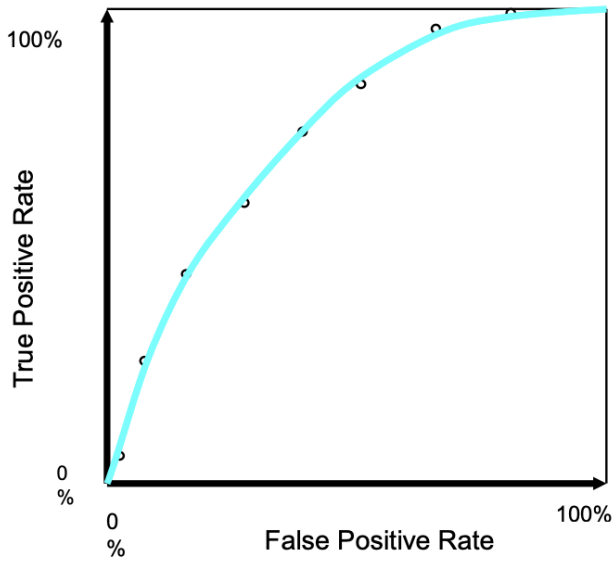
The distributions overlap completely

ROC curve comparison

A good test:



A poor test:



Syntax and Grammar

- Goal of syntactic theory
  - "explain how people combine words to form sentences and how children attain knowledge of sentence structure"
- Grammar
  - implicit knowledge of a native speaker
  - acquired without explicit instruction
  - minimally able to generate all and only the possible sentences of the language

Syntax in NLP

- Syntactic analysis often a key component in applications
  - Grammar checkers
  - Dialogue systems
  - Question answering
  - Information extraction
  - Machine translation
  - …

Two views of syntactic structure

- Constituency (phrase structure)
  - Phrase structure organizes words in nested constituents
- Dependency structure
  - Shows which words depend on (modify or are arguments of) which on other words

Constituency

- Basic idea: groups of words act as a single unit
- Constituents form coherent classes that behave similarly
  - With respect to their internal structure: e.g., at the core of a noun phrase is a noun
  - With respect to other constituents: e.g., noun phrases generally occur before verbs

Constituency structure

- **Phrase structure** organizes words into **nested constituents**
- Starting units: words are given a category: part-of-speech tags

  the, cuddly, cat, by, the, door

  Det, Adj, N, P, Det, N
- Words combine into phrases with categories

  the cuddly cat, by the door

  NP→Det Adj N PP→P NP
- Phrases can combine into bigger phrases recursively

  the cuddly cat by the door

  NP→ NP PP

Context-Free Grammars

- Terminals
  - We'll take these to be words (for now)
- Non-Terminals
  - The constituents in a language (e.g., noun phrase)
- Rules
  - Consist of a single non-terminal on the left and any number of terminals and non-terminals on the right

An Example Grammar

Grammar Rules			Examples
S	$\rightarrow$	*NP VP*	I + want a morning flight
NP	$\rightarrow$	*Pronoun*	I
	|	*Proper-Noun*	Los Angeles
	|	*Det Nominal*	a + flight
Nominal	$\rightarrow$	*Nominal Noun*	morning + flight
	|	*Noun*	flights
VP	$\rightarrow$	*Verb*	do
	|	*Verb NP*	want + a flight
	|	*Verb NP PP*	leave + Boston + in the morning
	|	*Verb PP*	leaving + on Thursday
PP	$\rightarrow$	*Preposition NP*	from + Los Angeles

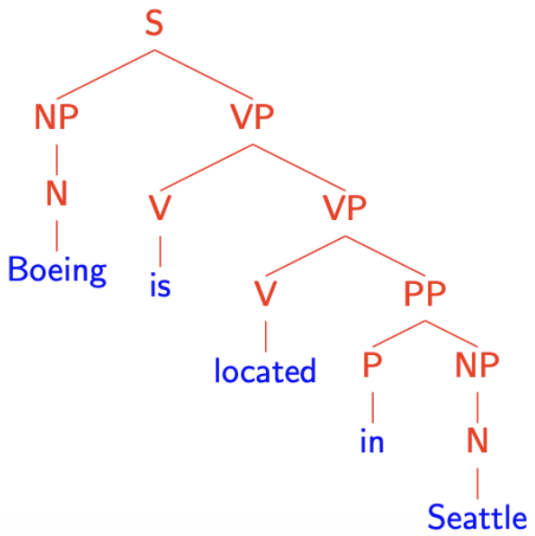
Syntactic parsing

- Syntactic parsing is the task of recognizing a sentence and assigning a structure to it.

Input:

Beoing is located in Seattle.

Output:

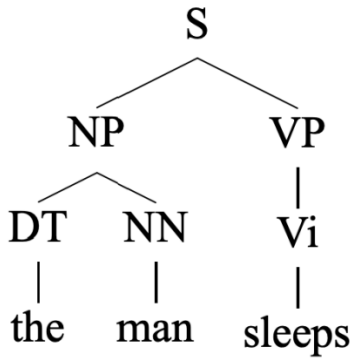


(Left-most) Derivations

- Given a CFG G, a left-most derivation is a sequence of strings $s_1, s_2, \ldots, s_n$, where
  - $s_1 = S$
  - $s_n \in \Sigma^*$: all possible strings made up of words from Σ
  - Each s_i for $i = 2,\ldots, n$ is derived from s_{i-1} by picking the left-most non-terminal X in s_{i-1} and replacing it by some β where $X \rightarrow \beta \in R$
- s_n: yield of the derivation

(Left-most) Derivations

- $s_1 = \text{S}$
- $s_2 = \text{NP VP}$
- $s_3 = \text{DT NN VP}$
- $s_4 = \text{the NN VP}$
- $s_5 = \text{the man VP}$
- $s_6 = \text{the man Vi}$
- $s_7 = \text{the man sleeps}$



$R =$

S	→	NP	VP
VP	→	Vi	
VP	→	Vt	NP
VP	→	VP	PP
NP	→	DT	NN
NP	→	NP	PP
PP	→	IN	NP

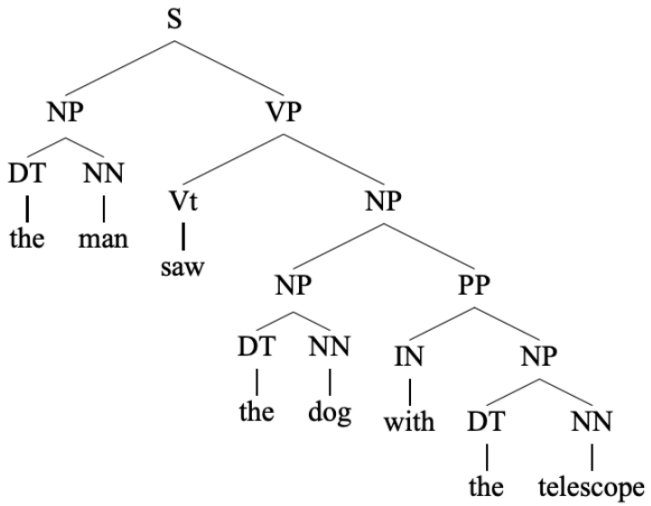
Vi	→	sleeps
Vt	→	saw
NN	→	man
NN	→	woman
NN	→	telescope
NN	→	dog
DT	→	the
IN	→	with
IN	→	in

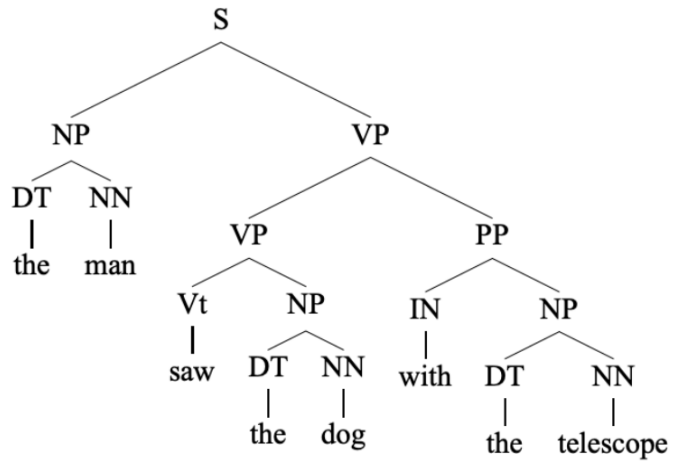
A derivation can be represented as a parse tree!

- A string $s \in \Sigma^*$ is in the language defined by the CFG if there is at least one derivation whose yield is s
- The set of possible derivations may be finite or infinite

Ambiguity

- Some strings may have more than one derivations (i.e. more than one parse trees!).





Tree Bank

The rise of annotated data

Starting off, building a treebank seems a lot slower and less useful than writing a grammar (by hand)

But a treebank gives us many things

- Reusability of the labor
  - Many parsers, part-of-speech taggers, etc. can be built on it
  - Valuable resource for linguistics
- Broad coverage, not just a few intuitions
- Frequencies and distributional information
- A way to evaluate NLP systems

Treebanks

- Standard setup (WSJ portion of Penn Treebank):
  - 40,000 sentences for training
  - 1,700 for development
  - 2,400 for testing

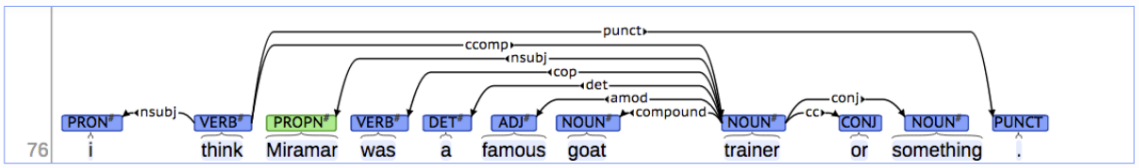
- Why building a treebank instead of a grammar?
  - Broad coverage
  - Frequencies and distributional information
  - A way to evaluate systems

Tree Bank

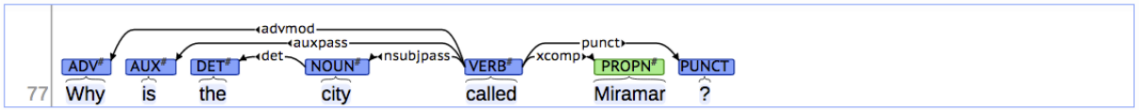
The rise of annotated data & Universal Dependencies treebanks

Brown corpus (1967; PoS tagged 1979); Lancaster-IBM Treebank (starting late 1980s);
Marcus et al. 1993, The Penn Treebank, *Computational Linguistics;*
Universal Dependencies: http://universaldependencies.org/

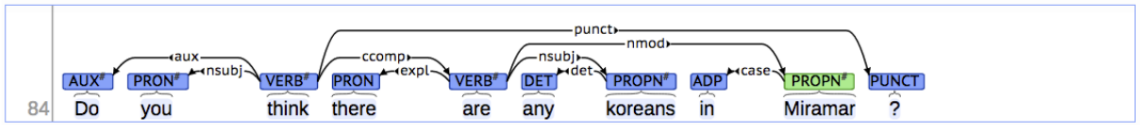
[context] [conllu]



[context] [conllu]

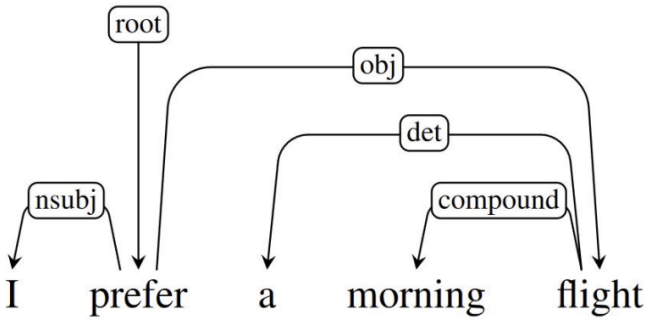


[context] [conllu]



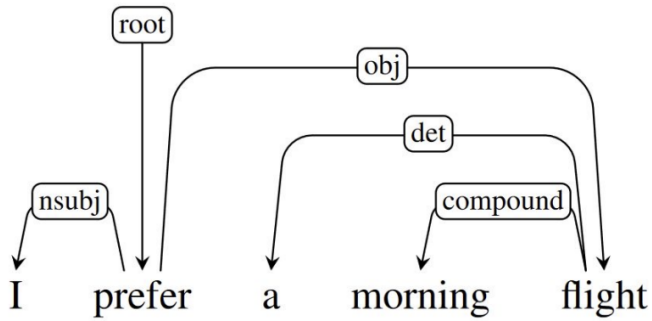
So what is dependency grammar?

- Directed binary grammatical relations between the words
- This direct encoding of the relationship between the predicates and their arguments (e.g., *prefer* takes *I* and *flight* as its arguments) is one of the reasons dependency grammar is more popular than constituency grammar in NLP!



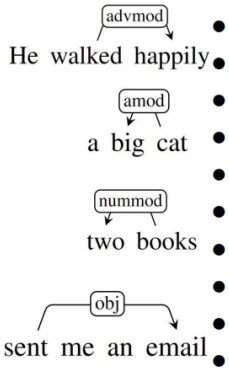
So what is dependency grammar?

- **Arcs** go from **heads** to **dependents**
- **Exactly 1 incoming edge** for all tokens! (but can have many **outgoing** ones)
- **Root** is the head of the entire structure
- Especially useful for languages with **free word order** (constituency grammar is less suited to those)

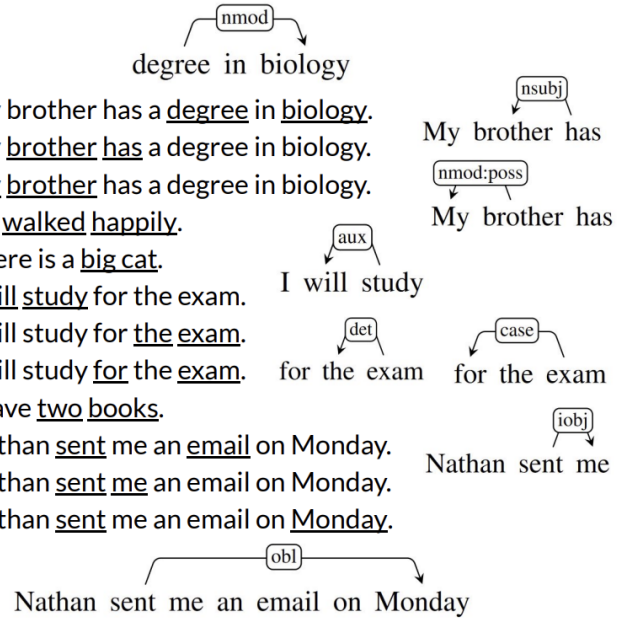


Dependency relations

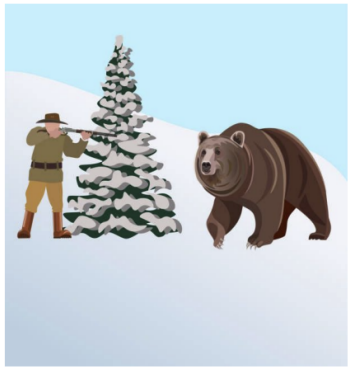
- advmod: adverb modifier
- amod: adjective modifier
- aux: auxiliary
- case: case marker
  (think of this as object of preposition)
- det: determiner
- iobj: indirect object
- nmod: noun modifier
- nmod:poss: possessive modifier
- nsubj: subject
- nummod: number modifier
- obj: direct object
- obl: oblique case ("prepositionally marked nominals functioning adverbially")
- root: root

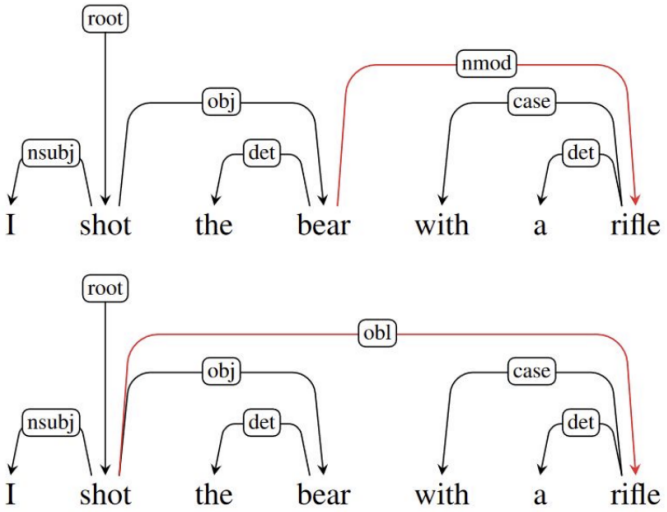


- My brother has a degree in biology.
- My brother has a degree in biology.
- My brother has a degree in biology.
- He walked happily.
- There is a big cat.
- I will study for the exam.
- I will study for the exam.
- I will study for the exam.
- I have two books.
- Nathan sent me an email on Monday.
- Nathan sent me an email on Monday.
- Nathan sent me an email on Monday.



Structural ambiguity (again!)

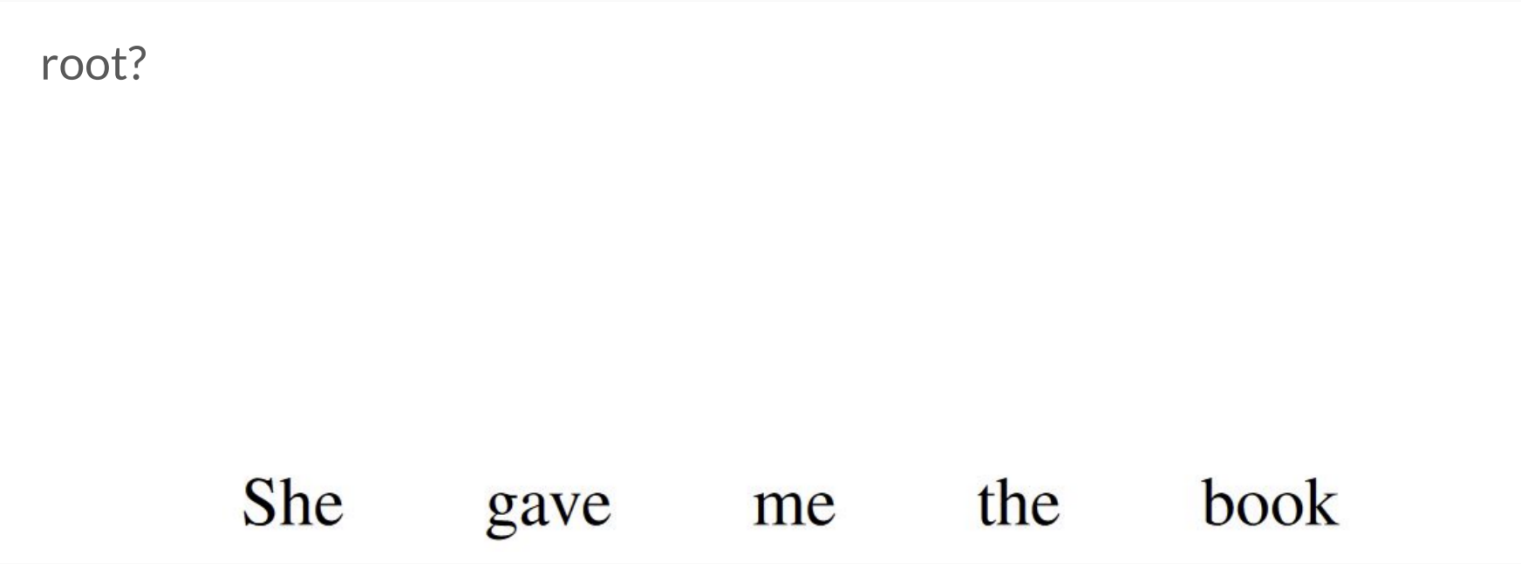




Images from: https://www.cs.utexas.edu/~dnp/frege/subsection-178.html
https://www.istockphoto.com/vector/hunter-shoots-a-bear-gm930143498-255034331

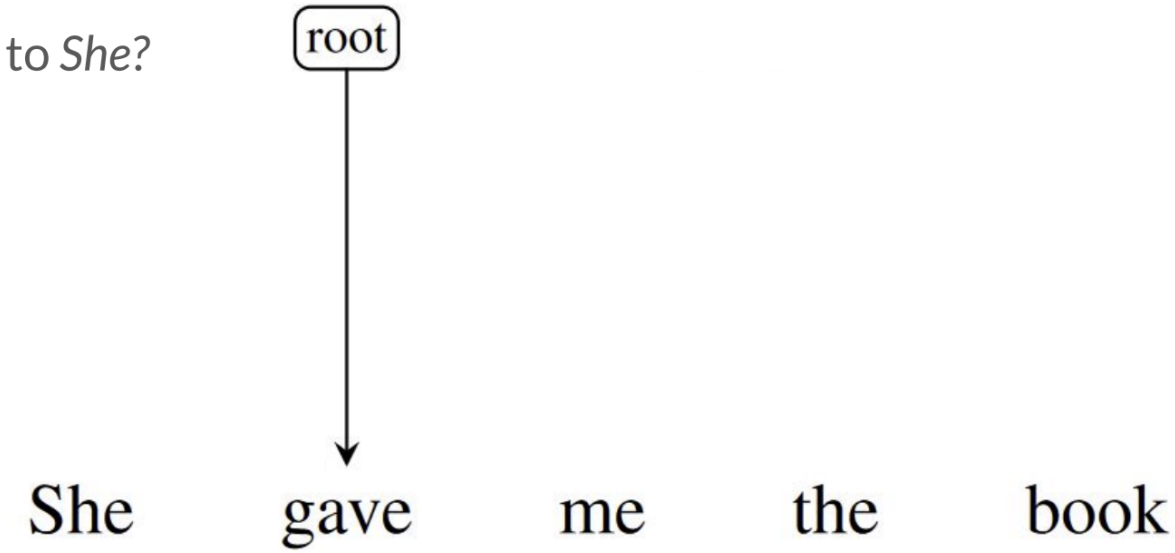
Dependency Parsing

root?



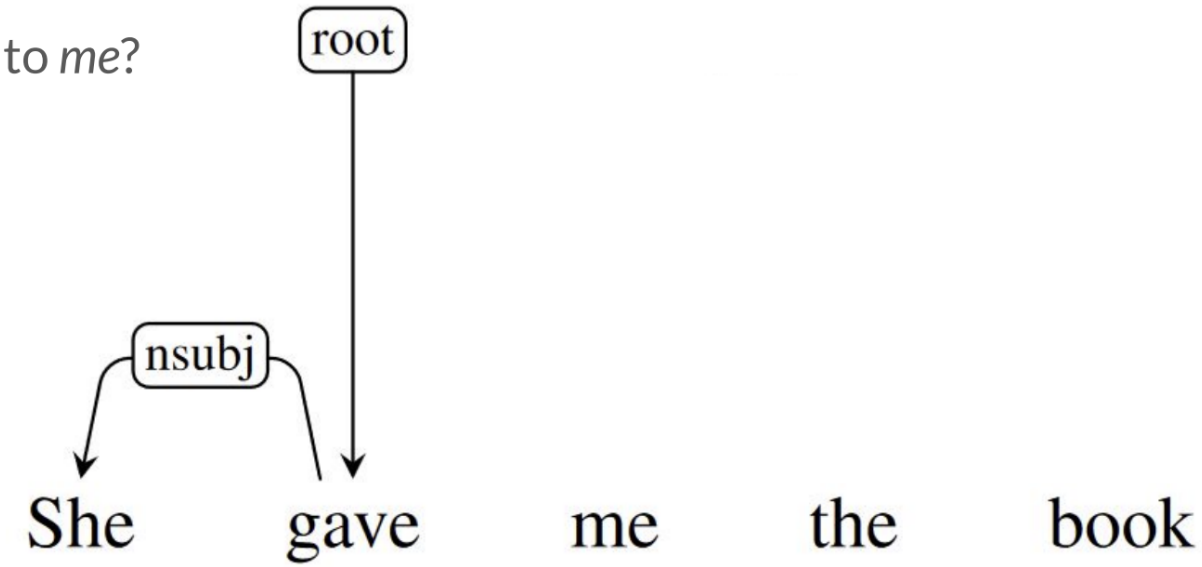
Dependency Parsing

Relation to *She*?



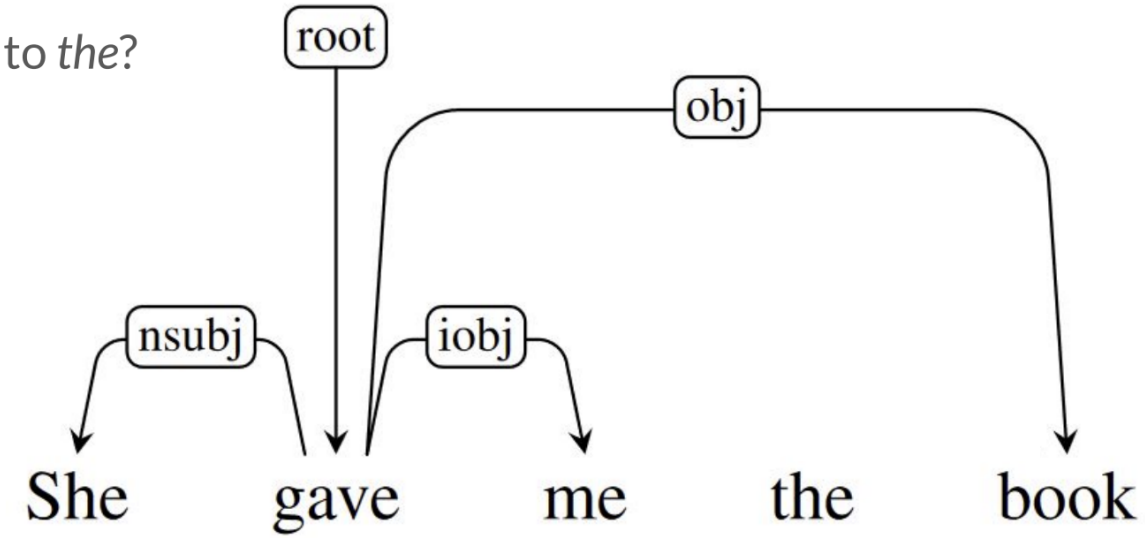
Let's try one!

Relation to *me*?

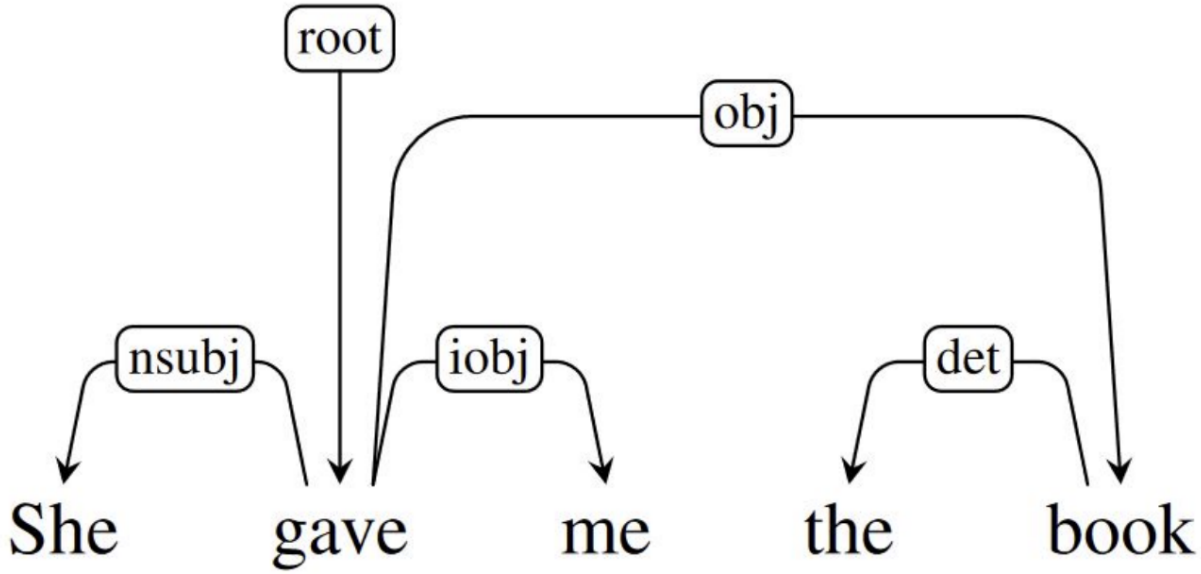


Let's try one!

Relation to *the*?



Let's try one!



3. Methods of Dependency Parsing

1. Dynamic programming

   Eisner (1996) gives a clever algorithm with complexity $O(n^3)$, by producing parse items with heads at the ends rather than in the middle
2. Graph algorithms

   You create a Minimum Spanning Tree for a sentence

   McDonald et al.'s (2005) $O(n^2)$ MSTParser scores dependencies independently using an ML classifier (he uses MIRA, for online learning, but it can be something else)

   Neural graph-based parser: Dozat and Manning (2017) et seq. – very successful!
3. Constraint Satisfaction

   Edges are eliminated that don't satisfy hard constraints. Karlsson (1990), etc.
4. "Transition-based parsing" or "deterministic dependency parsing"

   Greedy choice of attachments guided by good machine learning classifiers

   E.g., MaltParser (Nivre et al. 2008). Has proven highly effective. And fast.

Transition-based Parsing

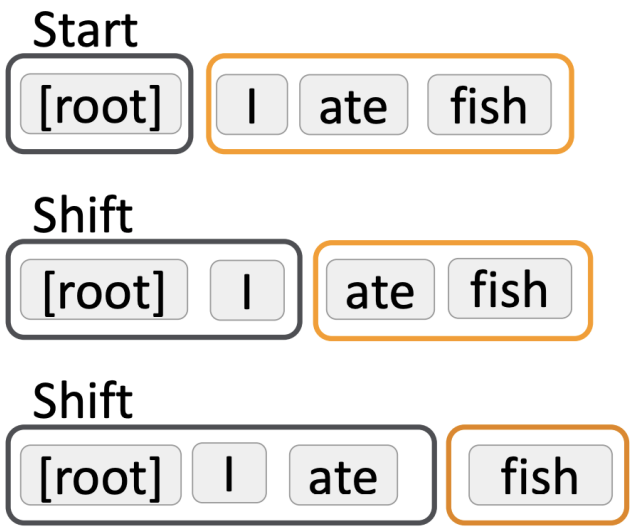
- Process words from left to right, deciding if the two words should be attached
- Build a dependency parse using a stack and buffer
- **Input buffer**: words of the sentence
- **Stack**: to manipulate the words
- **Dependency relations**: list of relations that culminate in the dependency parse

Transition-based Parsing

Arc-standard transition-based parser

(there are other transition schemes …)

Analysis of "I ate fish"



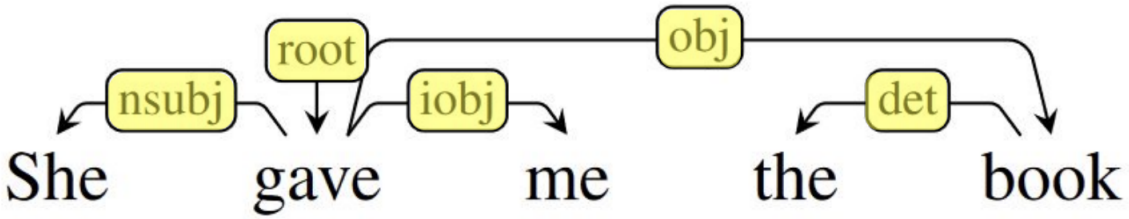
Start: σ = [ROOT], β = w_1, …, w_n , A = ∅

1. Shift σ, w_i|β, A ➔ σ|w_i, β, A
2. Left-Arc$_r$ σ|w_i|w_j, β, A ➔ σ|w_j, β, AU{$r(w_j,w_i)$}
3. Right-Arc$_r$ σ|w_i|w_j, β, A ➔ σ|w_i, β, AU{$r(w_i,w_j)$}

Finish: σ = [w], β = ∅

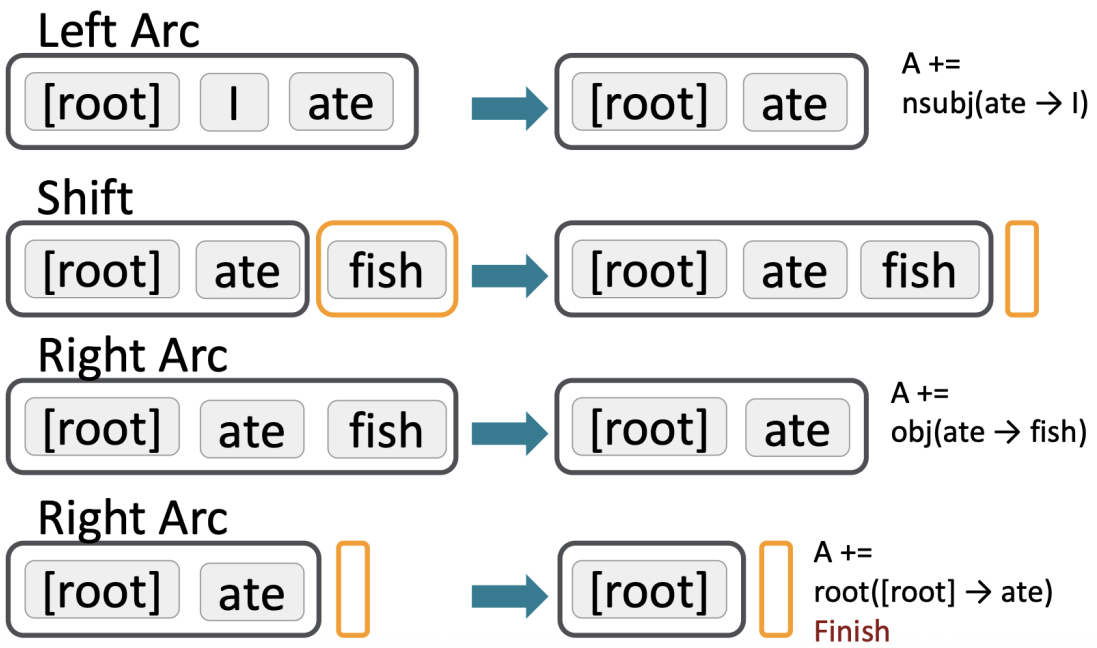
Transition-based Parsing

Step	Stack	Input Buffer	Action	Relation Added
0	[root]	[She, gave, me, the, book]	S	
1	[root, She]	[gave, me, the, book]	S	
2	[root, She, gave]	[me, the, book]	LA	(She ← gave)
3	[root, gave]	[me, the, book]	S	
4	[root, gave, me]	[the, book]	RA	(gave → me)
5	[root, gave]	[the, book]	S	
6	[root, gave, the]	[book]	S	
7	[root, gave, the, book]	[]	LA	(the ← book)
8	[root, gave, book]	[]	RA	(gave → book)
9	[root, gave]	[]	RA	(root → gave)
10	[root]	[]	Done	



Arc-standard transition-based parser

Analysis of "I ate fish"



Nota bene:
In this example I've at each step made the "correct" next transition.
But a parser has to work this out – by exploring or inferring!

A = { nsubj(ate → I),
obj(ate → fish)
root([root] → ate) }

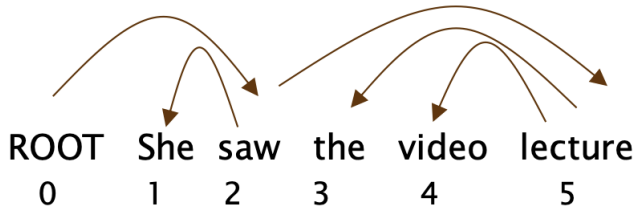
Transition-based Parsing

MaltParser [Nivre and Hall 2005]

- We have left to explain how we choose the next action 🤷
  - Answer: Stand back, I know machine learning!
- Each action is predicted by a discriminative classifier (e.g., softmax classifier) over each legal move
  - Max of 3 untyped choices (max of |R| × 2 + 1 when typed)
  - Features: top of stack word, POS; first in buffer word, POS; etc.
- There is NO search (in the simplest form)
  - But you can profitably do a beam search if you wish (slower but better):
    - You keep *k* good parse prefixes at each time step
- The model's accuracy is *a bit* below the state of the art in dependency parsing, but
- It provides **very fast linear time parsing**, with high accuracy – great for parsing the web

Evaluation

Evaluation of Dependency Parsing: (labeled) dependency accuracy



$$\text{Acc} = \frac{\text{\# correct deps}}{\text{\# of deps}}$$

UAS = 4 / 5 = 80%

LAS = 2 / 5 = 40%

Gold			
1	2	She	nsubj
2	0	saw	root
3	5	the	det
4	5	video	nn
5	2	lecture	obj

Parsed			
1	2	She	nsubj
2	0	saw	root
3	4	the	det
4	5	video	nsubj
5	2	lecture	ccomp